

FOSSEE IITB 2026

Screening Task — Documentation Report

Parametric 3D CAD Model of a Steel Girder Bridge (pythonOCC)

Submitted by	Mansi Pillai
Department	Mechanical Engineering
Institute	Institute of Engineering and Technology, DAVV, Indore
Task Area	Software Development — Parametric 3D CAD (pythonOCC)
Tool / Library	pythonOCC (pythonocc-core 7.x), Python 3.9
Date	12 April 2026

1. Introduction

This report documents the development of a parametric 3D CAD model of a short-span steel girder bridge, submitted as part of the FOSSEE IITB 2026 screening task under the Software Development area. The model is implemented entirely in Python using the pythonOCC (pythonocc-core) library, which provides Python bindings to the OpenCASCADE geometry kernel.

Osdag is a free and open-source software developed at IIT Bombay for the design and detailing of steel structures. This task contributes to Osdag's bridge module by providing a reusable, parametric 3D geometry engine for short-span steel girder bridges. The deliverable is a single runnable script — `bridge_model.py` — that assembles a complete bridge model from modular component factories.

2. Objectives

- Develop a parametric, modular 3D CAD model of a short-span steel girder bridge.
- Implement reusable component factory functions for each bridge element.
- Correctly position all components in 3D space using parametric transforms.
- Display concrete components semi-transparently to expose internal rebar.
- Export the assembled model to STEP and BREP formats.
- Include tests, CLI parameter overrides, and comprehensive documentation.

3. Coordinate System

A consistent right-handed coordinate system is used throughout the model, following the specification exactly:

Axis	Direction	Description
X	Longitudinal	Along the bridge span (0 to <code>span_length_L</code>)
Y	Transverse	Across the deck width (centred at $Y = 0$)
Z	Vertical	Upward direction
Origin	—	Centre of span at bottom face of deck slab ($Z = 0$)

The Z-level stack from top to bottom is: deck top (`+deck_thickness`), deck bottom / girder top ($Z=0$), girder bottom (`-girder_section_d`), pier cap, pier column, pile cap, and pile tips at the deepest negative Z.

4. Bridge Components and Architecture

The bridge model consists of a superstructure (girders, cross-frames, deck slab, deck rebar, parapets) and a substructure (pier cap, pier column with rebar cage, pile cap with rebar, and piles). Every component is created by a dedicated factory function and positioned by a shared `translate()` utility.

4.1 Superstructure

I-Section Girders. The main longitudinal girders are built by `create_i_section(d, bf, tf, tw, length)`. Three boxes (bottom flange, top flange, web) are Boolean-fused into a single solid. The section is centred in Y and positioned so its top face aligns with $Z=0$ (deck soffit). Girder spacing is controlled by `girder_centroid_spacing`.

Cross-Frames. Transverse stiffening plates are placed at regular intervals along the span by `build_crossframes()`. The depth is automatically clamped to the web height to prevent clipping into the flanges.

Deck Slab. A single rectangular prism spanning the full deck width and span length, sitting from $Z=0$ to $Z=+deck_thickness$. Set semi-transparent at `opacity=0.6` to expose the rebar grid below.

Deck Rebar Grid. Created by `create_rebar_grid_for_deck()`. Longitudinal bars run along X, spaced at `rebar_spacing_longitudinal` in Y. Transverse bars run along Y, spaced at `rebar_spacing_transverse` in X. Both layers are offset from the deck bottom by `rebar_cover` to respect concrete cover requirements.

Parapets. Rectangular parapet walls sit on top of both longitudinal edges of the deck slab. Symmetric positioning is enforced: left parapet at $Y=-deck_width/2$, right parapet at $Y=+deck_width/2-parapet_thickness$.

4.2 Substructure

Pier Cap (Hammerhead). Created by `create_trapezoidal_pier_cap()` using a `ThruSections` loft between two rectangular wires — narrow at the bottom (`pier_cap_bot_width` in Y) and wide at the top (`pier_cap_top_width` in Y). The along-span dimension is `pier_cap_span_length` (~800mm), correctly kept separate from the transverse `deck_width`. This produces the characteristic hammerhead shape seen in real bridge piers.

Pier Column. A vertical cylinder created by `create_circular_pier()`. A full rebar cage is generated by `create_pier_rebar_cage()`, consisting of longitudinal bars equally spaced around the circumference and dodecagonal stirrups at regular vertical intervals. The cage is displayed in red inside the semi-transparent pier concrete.

Pile Cap and Piles. The pile cap is a rectangular prism. Below it, a parametric grid of piles is generated — the number of piles is controlled by `n_piles_per_cap` (default 4, i.e. 2×2). The grid spacing is set by `pile_spacing`. A two-way rebar mat is added at the base of the pile cap.

5. Code Structure

The script is organised into four clearly separated layers, each dependent only on the layer above:

Layer	Contents	Purpose
1 — Parameters	Named variables at top of file	Single place to change all dimensions
2 — Factories	<code>create_*</code> functions	Return one TopoDS solid per component
3 — Assembly	<code>build_*</code> functions	Position and group components in 3D
4 — Display / Export	<code>assemble_bridge()</code> , <code>main()</code>	Render and optionally export

5.1 Factory Functions

Function	Returns	Key OCC API
create_i_section(d,bf,tf,tw,length)	I-beam solid	MakeBox + BRepAlgoAPI_Fuse
create_rectangular_prism(l,b,h)	Box solid	BRepPrimAPI_MakeBox
create_circular_pier(dia,h)	Cylinder solid	BRepPrimAPI_MakeCylinder
create_trapezoidal_pier_cap(...)	Lofted solid	BRepOffsetAPI_ThruSections
create_pile(dia,length)	Cylinder solid	BRepPrimAPI_MakeCylinder
create_pile_cap(l,w,d)	Box solid	BRepPrimAPI_MakeBox
create_rebar_grid_for_deck(...)	List of cylinders	MakeCylinder with gp_Ax2
create_pier_rebar_cage(...)	Bars + stirrups	MakeCylinder + MakePrism

5.2 Assembly Functions

Function	Description
build_girders()	Creates n_girders I-sections, spaces along Y, hangs below Z=0
build_crossframes()	Places cross-frame plates in web zone at X intervals
build_deck()	Single deck prism, centred in Y, sitting at Z=0
build_piers_and_pilecaps()	Iterates pier_x_locations, calls _build_single_pier()
build_parapets()	Two parapet walls on deck edges, toggled by show_parapets
assemble_bridge()	Combines all into TopoDS_Compound with colour metadata

6. Parameter Reference

All parameters are exposed as named variables at the top of `bridge_model.py` and can also be overridden at runtime via CLI flags (see Section 7). Default values match the example dimensions given in the problem statement.

Parameter	Default	Unit	Description
<code>span_length_L</code>	12000	mm	Total clear span length
<code>n_girders</code>	3	—	Number of longitudinal girders (min 3)
<code>girder_centroid_spacing</code>	3000	mm	Centre-to-centre girder spacing
<code>deck_overhang</code>	500	mm	Deck overhang beyond outer girders
<code>deck_width</code>	7000	mm	Derived: $(n-1)*\text{spacing} + 2*\text{overhang}$
<code>girder_section_d</code>	900	mm	I-section total depth
<code>girder_section_bf</code>	300	mm	Flange width
<code>girder_section_tf</code>	16	mm	Flange thickness
<code>girder_section_tw</code>	10	mm	Web thickness
<code>deck_thickness</code>	200	mm	Deck slab thickness
<code>concrete_opacity</code>	0.6	—	Transparency of concrete (0=solid)
<code>pier_diameter</code>	800	mm	Pier column diameter
<code>pier_height</code>	3000	mm	Pier column height
<code>pier_cap_top_width</code>	3500	mm	Hammerhead top transverse width
<code>pier_cap_bot_width</code>	1000	mm	Hammerhead bottom transverse width
<code>pier_cap_depth</code>	600	mm	Pier cap vertical depth
<code>pier_cap_span_length</code>	800	mm	Pier cap along-span bearing length
<code>pile_diameter</code>	400	mm	Pile diameter
<code>pile_length</code>	5000	mm	Pile length
<code>n_piles_per_cap</code>	4	—	Piles per cap (perfect square, e.g. $4=2\times 2$)
<code>pile_spacing</code>	600	mm	Pile centre-to-centre spacing
<code>pile_cap_length</code>	2200	mm	Pile cap length along X
<code>pile_cap_width</code>	1200	mm	Pile cap width along Y
<code>pile_cap_depth</code>	600	mm	Pile cap depth
<code>rebar_main_diameter</code>	16	mm	Longitudinal rebar diameter
<code>rebar_transverse_diameter</code>	8	mm	Transverse/stirrup rebar diameter
<code>rebar_spacing_longitudinal</code>	150	mm	Spacing of longitudinal bars
<code>rebar_spacing_transverse</code>	200	mm	Spacing of transverse bars
<code>rebar_cover</code>	40	mm	Concrete cover

Parameter	Default	Unit	Description
rebar_visible	True	—	Toggle rebar display
crossframe_spacing	3000	mm	Cross-frame spacing along span
show_parapets	True	—	Toggle parapet walls
save_step	False	—	Export STEP file on run
save_brep	False	—	Export BREP file on run

7. Command-Line Interface

All key parameters can be overridden at runtime without editing the source file. Derived values (deck_width, girder_length, pier_x_locations) are automatically recomputed after CLI parsing.

Flag	Type	Example	Effect
--span	float	--span 15000	Sets span_length_L
--n-girders	int	--n-girders 4	Sets n_girders (min 3)
--girder-spacing	float	--girder-spacing 2500	Sets girder_centroid_spacing
--crossframe-spacing	float	--crossframe-spacing 2000	Sets crossframe_spacing
--opacity	float	--opacity 0.4	Sets concrete_opacity
--no-rebar	flag	--no-rebar	Hides all rebar
--no-parapets	flag	--no-parapets	Hides parapet walls
--save-step	flag	--save-step	Exports STEP file
--save-brep	flag	--save-brep	Exports BREP file

Example: `python bridge_model.py --span 15000 --n-girders 4 --opacity 0.3 --save-step`

8. Export Capabilities

The full bridge compound (all components fused into a TopoDS_Compound) can be exported to two interoperable formats:

STEP (.step) — Industry-standard exchange format supported by all major CAD tools (SolidWorks, AutoCAD, FreeCAD, CATIA). Enabled with --save-step or by setting save_step = True.

BREP (.brep) — OpenCASCADE native format for lossless geometry preservation. Enabled with --save-brep or by setting save_brep = True.

9. Testing

A companion test file test_bridge.py is provided. It runs 11 tests without opening a display window, covering all component factories, all assembly functions, and parameter consistency checks. Run with:

```
python test_bridge.py
```

Test	What it checks
test_i_section_created	create_i_section() returns a non-None solid
test_cylinder_created	create_circular_pier() returns a non-None solid
test_box_created	create_rectangular_prism() returns a non-None solid
test_trapezoidal_cap	create_trapezoidal_pier_cap() loft succeeds
test_girder_count	build_girders() returns exactly n_girders shapes
test_deck_created	build_deck() returns a non-None solid
test_rebar_deck	create_rebar_grid_for_deck() returns > 0 bars
test_piers_created	build_piers_and_pilecaps() returns > 0 parts
test_crossframes	build_crossframes() returns > 0 frames
test_deck_width_positive	deck_width > 0 (parameter sanity)
test_deck_width_formula	deck_width == (n-1)*spacing + 2*overhang

10. Known Limitations and Future Work

Stirrup rendering. Dodecagonal stirrups are generated but may appear faint at the full bridge scale due to their small cross-section relative to the pier height. A future improvement would use a swept torus for the stirrup solid.

Boolean fuse performance. The three-part Boolean fuse for each I-section is correct but could be replaced with a single-extrusion approach using BRepPrimAPI_MakePrism on a 2D profile wire for better performance at large n_girders values.

IFC export. The spec mentions optional IFC export. This was not implemented in the current version as it requires the ifcopenshell library which was outside the scope of this screening task. It is noted as a clear next step.

Abutments. End abutments at X=0 and X=span_length_L are not modelled. These would be the next structural element to add for a complete bridge model.

Deck segmentation. deck_slab_segment_length parameter is defined but not yet used — the deck is currently one prism. Segmentation would improve realism for longer spans.

11. Submission Checklist

Item	File / Link	Status
Python script	bridge_model.py	Included in ZIP
Test file	test_bridge.py	Included in ZIP
Starter files	first.py, second.py	Included in ZIP
Documentation	FOSSEE_Report_Mansi_Pillai.pdf	This document

Item	File / Link	Status
Video demo	https://youtu.be/LPC0olrSCh4	Uploaded
GitHub repo	https://github.com/smansipillai009/fossee-bridge-model.git	osdag-admin added
ZIP archive	fossee_bridge_model.zip	Submitted

This model was developed as a screening task submission for FOSSEE IITB 2026, Software Development track — Parametric 3D CAD Model of a Steel Girder Bridge (pythonOCC). All code is original work. The pythonOCC library and OpenCASCADE geometry kernel are used under their respective open-source licences.